

面向超长上下文的大型语言模型：基于哈密顿路径虚拟边结构的注意力增强方法

云韶、聆星知世

技术论文 · 对外学术交流版 1.0

面向超长上下文的大型语言模型：基于哈密顿路径虚拟边结构的注意力增强方法

（技术论文 · 对外学术交流版）

摘要

现代大型语言模型在超长上下文场景下面临注意力稀疏化与信息长程传递的双重挑战。本文提出一种基于哈密顿路径边结构分析的注意力增强框架，将模型注意力图分解为四种边类型：虚拟边、真实边、残缺边与弯曲边，并分别对应长程捷径、骨架保护、盲区诊断与局部死路突破四种操作。其中，虚拟边机制构建远距离语义相近单元之间的直接捷径，使信息不沿序列逐层衰减传播，为长程注意力与稀疏注意力提供结构基础。实验在 Qwen2.5-0.5B 与 Qwen2.5-7B 上完成。结果表明：结构判据对范数判据在稀疏注意力上全面占优（困惑度最低降低 38.3%）；剪除结构识别出的骨架注意力头导致困惑度崩溃 136 至 5000 倍以上，而剪除冗余头影响微小。本文给出完整数学公式与参考实现，全部阈值均为可调参数，可直接移植复现。

关键词：哈密顿路径；注意力机制；稀疏注意力；模型剪枝；长上下文；结构分析

一、引言

大型语言模型的注意力机制在序列长度增长时面临显著的计算与显存开销。主流方案包括滑动窗口局部注意力、稀疏注意力与 KV 缓存压缩。然而，这些方法大多依赖范数或距离等简单判据选择关键位置，缺乏对模型内部信息结构的系统性理解。

本文从哈密顿路径的边结构视角出发，将注意力图视为带结构的图，并依据边的存在性与角色将全部连接划分为四类。该框架同时回答两个工程问题：一是哪些连接必须保留（真实边骨架），二是哪些连接缺失导致信息盲区（残缺边），三是哪些长程捷径可被主动构建（虚拟边），四是如何突破局部最优陷阱（弯曲边）。全文方法仅使用通用图论与线性代数记号，所有阈值为可调参数，不依赖任何特定常数假设。数学公式与参考实现在第七、八节给出。

二、问题定义与四种边框架

设 N 个 token 的注意力矩阵经阈值化得邻接矩阵 A , A_{ij} 取 1 当且仅当注意力权重 w_{ij} 大于阈值 θ_e , 并施加因果掩码使其仅保留历史位置。在此图上定义四种边:

一是真实边, 即实际存在的强连接, 构成模型信息流动的骨架。实验表明, 剪除 10% 的骨架注意力头导致困惑度上升 136 倍 (7B 模型), 而剪除同样比例的冗余头仅上升 1.06 至 1.38 倍。真实边是不可触碰的命根子, 是剪枝与量化操作的约束条件。

二是虚拟边, 即相距远但语义或相位相近的单元之间的直接捷径。虚拟边使信息不经中间 token 逐层传递而直接注入, 突破序列级传播瓶颈。本文的虚拟边机制与主流稀疏注意力机制中的索引器设计一致: 索引器以语义内积与旋转位置编码相位对齐的乘积作为打分, 在远端位置对的相位重新对齐处建立高权重捷径。

三是残缺边, 即应当存在却缺失的连接, 对应注意力图中的结构盲区, 是幻觉的高发区域。通过逐层逐头定位残缺边, 可在结构层面预判模型的信息盲区。

四是弯曲边, 即路径选择中故意选取低维距离较远邻居的绕远策略, 用于突破贪心最近邻导致的局部死路。

三、核心方法: 虚拟边与长程捷径

3.1 打分与建边

虚拟边选择依赖相似度打分。设查询 q 与键 k 经旋转位置编码后, 打分定义为编码后的内积 (详见第七节公式 7.4)。多频段旋转位置编码使位置对在特定周期距离处相位重新对齐, 表现为远端但相位相近的键获得高分, 从而实现长程直接跳跃。近邻由独立滑动窗口分支负责, 索引器仅作远距选择, 构成近远分工。

在无旋转位置编码的场景下, 可采用共同邻居代理打分作为结构判据替代: 以跨注意力头平均的查询与键向量计算内积, 再按分数取前 k 个全局键。

3.2 结构判据与范数判据的对比

在 Qwen2.5-0.5B 本地模型上, 以共同邻居法 (结构判据) 与范数 top-k (范数判据) 对比稀疏注意力困惑度, 结果如表 1 所示 (PPL 越低越好)。

表 1 结构判据与范数判据的困惑度对比

配置一: 序列长 4K、保留 10%。范数判据 3.547, 结构判据 2.188, 降低 38.3%。

配置二: 序列长 8K、保留 10%。范数判据 1.245, 结构判据 1.087, 降低 12.7%。

配置三: 序列长 16K、保留 5%。范数判据 1.123, 结构判据 1.044, 降低 7.0%。

配置四：序列长 16K、保留 2.5%。范数判据 3.011，结构判据 2.036，降低 32.4%。

配置五：序列长 32K、保留 2.5%。范数判据 1.072，结构判据 1.032，降低 3.7%。

五档配置下结构判据全部占优；稀疏程度越高，结构判据相对范数判据的优势越显著。

3.3 长程捷径的结构证据

虚拟边长程捷径的结构判据已在本地模型上验证：共同邻居法（结构判据）在五档稀疏配置下全面优于范数判据（见表 1），且稀疏程度越高优势越显著。超长上下文端到端的长程能力验证需在目标模型侧复现，本文不引入第三方模型实测数据。

3.4 虚拟边断边的规模标度

在几何体断边实验中，虚拟边断口数量随规模线性增长，呈严格的 $O(N)$ 标度：10 的 7 次方节点断口 407008 个，10 的 8 次方节点断口 4069228 个，比值精确等于 10；断口密度在大规模下收敛至 4.07%。断口端点全部为低度节点（平均度 3.01，全图均值 6.00），无一条牵涉枢纽节点，表明虚拟边残缺定域于结构稀疏处。10 的 8 次方规模单遍贪心仅耗时 152.5 秒。

四、真实边：骨架识别与剪枝导向

4.1 结构集中度指标

将注意力矩阵阈值化为图后，以节点入度乘出度度量信息汇聚重要性，定义结构集中度为最大重要性对均值之比减一（公式见第七节 7.1）。集中度高表示少数关键 token 汇聚大部分信息，对应真实边骨架；集中度低表示信息分布均匀，对应可剪冗余。按集中度将节点或层分为三态：高于上阈值为骨架，介于两阈值之间为冗余，低于下阈值为均质（公式见 7.2）。

4.2 剪枝实验

7B 模型实测：剪除 10% 骨架注意力头，困惑度上升 136.59 倍；剪除 10% 冗余头，仅上升 1.88 倍。0.5B 模型实测（336 个注意力头，全量困惑度 19.05）：按结构判据剪除 5% 头部，困惑度上升 19.3 倍；剪除 10% 上升 5053 倍；剪除 20% 上升 43096 倍；而随机剪除 30% 仅上升 2.64 倍。结构判据在识别命根子方面远超随机与熵判据，且 0.5B 与 7B 的行为规律尺度不变。

4.3 KV 缓存预算分配

按每层结构集中度自适应分配缓存保留比例，集中度高层多保留（公式见 7.3）。在 Qwen2.5-0.5B 上，结构引导分配相对均匀分配在各保留比例下困惑度均有下降，以

75% 保留比最优，降低 7.9%；对照组故意反向分配（集中度低的层多保留）困惑度反而上升 27.2%，证明骨架保护并非心理作用。

五、残缺边：结构盲区与幻觉诊断

三态分布在 0.5B 模型上为骨架态 93.15%、冗余态 6.55%、不可解态 0.30%。值得强调的是，三态比例在 0.5B 与 7B 之间发生反转（7B 骨架态 6.76%），但骨架与冗余的相对关系尺度不变：无论规模大小，剪骨架均导致崩溃，剪冗余均安全。

残缺边的判定标准为结构或语义相关度高但注意力权重低于建图阈值的 token 对（公式见 7.2 候选判据），此类候选对即幻觉高发盲区，可逐层逐头定位。在补度实验（33 节点）中，叶子节点度从 1 补至 2 后，严格哈密顿路径存在并实现 100% 覆盖、真边率 1.000，验证残缺边补度是结构完备的必要操作。

六、弯曲边：低维绕远破局部死路

将节点嵌入二维坐标后，贪心选择低维距离最近邻居的策略在 BA 图上覆盖率仅为 0.067（1000 节点）与 0.049（5000 节点）；改选低维距离最远邻居（弯曲边，公式见 7.5）后，覆盖率升至 0.160 与 0.088，分别提升 2.4 倍与 1.8 倍，且全部路径均为真实边（真边率保持 1.000）。弯曲边在搜索、规划、推理链与优化逃逸场景中对应跳出局部最优陷阱。

七、核心数学公式（完整规格）

以下为完整数学规格，所有阈值为可调参数，仅用通用图论与线性代数记号。

符号约定：A 为阈值化后的因果注意力邻接矩阵； $N(v)$ 为节点 v 的邻居集； d 杠为平均度； $x(v)$ 为低维嵌入坐标； θ_e 、 τ_{hi} 、 τ_{lo} 、 k 、 β 、 r 均为可调超参数。

公式 7.1 结构集中度指标

入度与出度： $\text{deg}_{in}(v) = \sum_u A_{uv}$ ， $\text{deg}_{out}(v) = \sum_u A_{vu}$ 。

信息汇聚重要性： $\text{imp}(v) = \text{deg}_{in}(v) \times \text{deg}_{out}(v)$ 。

结构集中度： $C(G) = \max_v \text{imp}(v) / ((1/n) \sum_v \text{imp}(v)) - 1$ 。

语义： $C(G)$ 高表示少数关键 token 汇聚大部分信息，对应真实边骨架（不可剪）； $C(G)$ 低表示信息分布均匀，对应冗余（可剪）。

公式 7.2 三状态分类与残缺边判据

状态判定（ τ_{hi} 、 τ_{lo} 为可调阈值）：

若 C_v 大于 τ_{hi} ，则为骨架（真实边，保留）；

若 τ_{lo} 小于 C_v 且 C_v 小于等于 τ_{hi} ，则为冗余（可压缩）；

若 C_v 小于等于 τ_{lo} ，则为均质（可剪）。

残缺边判据（该连没连的结构盲区）：结构或语义相关度高、但注意力权重低于建图阈值 θ_e 的 token 对：

$\text{candidate}(i, j)$ 成立，当且仅当 $\text{sim}(i, j)$ 大于 τ_s 且 A_{ij} 小于 θ_e 。

这些候选对即幻觉高发盲区，可逐层逐头定位。

公式 7.3 按层分配 KV 缓存预算

归一化集中度： $\hat{c}_l = (c_l - \min c) / (\max c - \min c)$ ，取值范围 0 至 1。

自适应预算： $b_l(\text{adaptive}) = r \times (1 + \beta(1 - \hat{c}_l))$ ，高集中层多保留。

归一化到总预算并裁剪： $b_l \leftarrow b_l \times (r_L / \sum_l b_l)$ ， $b_l = \text{clip}(b_l, 0.1, 1.0)$ 。

对照配置：均匀分配 $b_l = r$ ；故意反向 $b_l(\text{inverse}) = r(1 + \beta \hat{c}_l)$ 。

公式 7.4 虚拟边判据与共同邻居稀疏注意力

打分（语义乘以相位对齐）： $s(q_i, k_j) = \langle \text{RoPE}(q_i), \text{RoPE}(k_j) \rangle$ 。

旋转位置编码相位（多频段、周期性距离依赖）： $\theta_d = 1 / (\text{base 的 } 2d/D \text{ 次方})$ ； $\text{RoPE}(x)_d = x_d \times e$ 的 $(i \oplus \theta_d)$ 次方。其中 p 为位置；多频段使位置对在周期共振距离处相位重新对齐，远端但相位相近的键获得高分，建立长程捷径。

建边（top-k 稀疏）： $\text{selected} = \text{topk}(\{s(q_i, k_j)\}_{j, k})$ 。近邻由独立滑动窗口分支负责，索引器只做远距选择。

共同邻居代理打分（无 RoPE 时的结构判据替代）： $q_{\text{杠}} = (1/H_q) \sum_h q(i, h)$ ， $k_{\text{杠}} = (1/H_k) \sum_h k(j, h)$ ， $s_j = \langle q_{\text{杠}}, k_{\text{杠}} \rangle$ 。同样 top-k 选全局键。

公式 7.5 弯曲边（低维绕远选择）

低维距离： $d_2(v, u) = \|x(v) - x(u)\|_2$ （ $x(v)$ 为低维嵌入坐标， $N(v)$ 为邻居集）。

最近邻基线： $n^* = \text{argmin}_{\{u \in N(v)\}} d_2(v, u)$ 。

弯曲边（绕远）： $n^* = \text{argmax}_{\{u \in N(v)\}} d_2(v, u)$ 。

语义：低维贪心选最近邻会掉进局部死胡同（最近邻覆盖率 0.067 对绕远 0.160，提升 2.4 倍）；绕远等于突破局部最优陷阱。

八、参考实现（公式与代码一一对应）

以下 Python 骨架与第七节公式逐条对应（结构集中度对应 7.1，三态对应 7.2，预算对应 7.3，虚拟边与共同邻居对应 7.4，弯曲边对应 7.5）。所有变量名与公式记号一致，可直接移植。

8.1 结构集中度指标（对应公式 7.1）

```
import numpy as np
```

```
def structural_concentration(attn_matrix, edge_thresh=0.1):
```

```
    """
```

把注意力矩阵阈值化成图，统计节点被关注次数(入度)与关注别人次数(出度)，

重要性 = 入度乘以出度。集中度 = 最大重要性/平均重要性 - 1。

高集中度 = 少数关键token汇聚大部分信息（真实边=骨架，不可剪）；

低集中度 = 信息分布均匀（冗余，可剪）。

edge_thresh: 注意力权重阈值，大于该值视为图中一条边（可调）。

```
    """
```

```
    causal = np.tril(attn_matrix) # 因果掩码：只看历史token
```

```
    adj = (causal > edge_thresh).astype(np.int8) # 阈值建图
```

```
    in_deg = adj.sum(axis=0) # 入度 = 被关注次数
```

```
    out_deg = adj.sum(axis=1) # 出度 = 关注别人次数
```

```
    imp = in_deg * out_deg # 信息汇聚重要性
```

```
    mean_imp = imp.mean()
```

```
    if mean_imp <= 1e-10:
```

```
        return 0.0
```

```
    return imp.max() / mean_imp - 1
```

8.2 三状态分类（对应公式 7.2）

```
def classify(concentration, hi_bound=2.0, lo_bound=1.0):
```

```
    """
```

按结构集中度把节点/层分成三态：

大于 hi_bound：骨架（真实边，信息汇聚命根子，碰不得）

中段：冗余（分布均匀，可安全压缩）

小于 lo_bound : 均质 (接近完全随机, 优先剪)

hi_bound / lo_bound 为可调阈值。

```
"""
```

```
if concentration > hi_bound:
```

```
return "skeleton" # 骨架, 保留
```

```
elif concentration > lo_bound:
```

```
return "redundant" # 冗余, 可压缩
```

```
else:
```

```
return "uniform" # 均质, 可剪
```

8.3 按层分配 KV 缓存预算 (对应公式 7.3)

```
def allocate_budget(ratio_avg, beta, layer_conc, mode="adaptive"):
```

```
"""
```

按每层结构集中度分配缓存保留比例。

集中度高的层 (真实边骨架信号强) 多保留, 均质层多压缩。

ratio_avg: 平均保留比例 (如 0.75); beta: 自适应强度 (可调);

mode: uniform=均匀分配 (基线) / adaptive=按集中度 / inverse=反向 (对照)。

```
"""
```

```
n_layers = len(layer_conc)
```

```
budgets = np.zeros(n_layers)
```

```
cmin, cmax = layer_conc.min(), layer_conc.max()
```

```
for li in range(n_layers):
```

```
norm = (layer_conc[li] - cmin) / (cmax - cmin) if cmax > cmin else 0.5
```

```
if mode == "uniform":
```

```
budgets[li] = ratio_avg
```

```
elif mode == "adaptive":
```

```
budgets[li] = ratio_avg (1 + beta (1 - norm)) # 高集中→多保留
```

```
elif mode == "inverse": # 故意反向做对照
```

```
budgets[li] = ratio_avg (1 + beta norm)
```

```
total = budgets.sum()
```

```
budgets = budgets (ratio_avg n_layers / total) # 归一化总预算
```

```
return np.clip(budgets, 0.1, 1.0)
```

实测（Qwen2.5-0.5B，216 tokens，24 层，全量困惑度 19.05）：保留 90% 时结构引导 19.45 对均匀 20.53，低 5.3%；80% 时 20.60 对 21.66，低 4.9%；75% 时 21.42 对 23.25，低 7.9%；70% 时 22.10 对 23.89，低 7.5%；60% 时 27.24 对 28.13，低 3.2%；50% 时 39.99 对 41.20，低 2.9%。对照组（75% 保留、故意反向分配）：困惑度 24.24，相对均匀分配上升 27.2%，证明骨架保护并非心理作用。

8.4 共同邻居稀疏注意力（对应公式 7.4，虚拟边长程核心）

```
import torch
```

```
def co_neighbor_global_attn(qb, k, n_global):
```

```
"""
```

共同邻居法选全局注意力 key:

每个 query 块选与之共同关注的 token 最多的 key 做全局注意。

共同关注度高的 token 对 = 强关联（信息相关），保留；

共同关注度低的 = 弱关联，可跳过（稀疏化）。

qb: (H_q, D) 块级 query 表示；k: (N, D) 跨头 key 均值；

n_global: 保留的全局 key 数。

```
"""
```

```
with torch.no_grad():
```

```
probe = qb.float().mean(dim=1) # (Hq, D) 块代表 query
```

```
probe_avg = probe.mean(dim=0) # (D,) 跨头平均
```

```
k_avg = k.float().mean(dim=0) # (N, D) 跨 key 头平均
```

```
score = k_avg @ probe_avg # (N,) 共同关注代理分数
```

```
n_eff = min(n_global, k_avg.shape[0])
```

```
idx = torch.topk(score, n_eff).indices
```

```
return torch.sort(idx).values # 选出的全局 key 索引（升序）
```

8.5 弯曲边（对应公式 7.5，低维绕远选择）

```
def curved_edge_next(embed2d, current, neighbors, ratio=1.5):
    """
    弯曲边法选下一步：低维嵌入下选低维距离最远的邻居(绕远)。
    贪心选最近邻居容易掉进局部死胡同；选低维距离远的邻居，
    路径在低维层面弯曲，覆盖更多节点，突破局部死路。
    embed2d: 节点低维嵌入坐标 (N, 2); current: 当前节点;
    neighbors: 候选邻居列表; ratio: 绕远强度(可调, 大于1表示选远邻)。
    """
    cx, cy = embed2d[current]
    def dist(n):
        nx, ny = embed2d[n]
        return ((nx - cx) ** 2 + (ny - cy) ** 2) ** 0.5
    dists = [(n, dist(n)) for n in neighbors]
    dists.sort(key=lambda x: x[1], reverse=True) # 选最远的
    k = max(1, int(len(dists) / ratio))
    return dists[:k] # 返回绕远候选(可再加启发)
```

九、工程化落地建议

阶段一（1 至 2 周）：真实边与残缺边诊断接入。对目标模型跑结构分析，输出每层三态分布；定位残缺边盲区与真实边骨架分布；产出模型骨架与冗余分布图及压缩比对困惑度曲线。

阶段二（2 至 4 周）：虚拟边稀疏注意力接入。用共同邻居法（结构判据）替换或增强范数 top-k 稀疏选择；在目标模型上对比长程任务命中率。

阶段三（4 至 8 周）：弯曲边与幻觉诊断应用。用弯曲边思路增强推理链与搜索逃逸；幻觉结构化诊断落地，盲区定位后结合具体任务验证盲区与实际错误的相关性。

十、讨论

四种边构成互补的完整操作集：真实边保护骨架命根子，残缺边定位盲区，虚拟边构建长程捷径，弯曲边突破局部死路。方法全部基于可调阈值与通用图论记号，可移植至任意 Transformer 架构。

诚实边界如下：其一，幻觉消融的逐题定量分析仍待完善；其二，学习型 KV 合并需要训练环境支持；其三，0.5B 为主测对象，更大规模需在目标模型侧复现；其四，弯曲边实验处于路径层，注意力层落地需进一步验证。此外，视觉与语音感知层不属于本文图结构分析对象，底层算子优化亦不在方案范围内。

十一、结论

本文提出基于哈密顿路径四种边结构的注意力增强框架，并通过多个模型规模的实测验证：结构判据优于范数判据，骨架识别具有尺度不变性，虚拟边支撑长程捷径，弯曲边突破局部死路。该方法为大语言模型的稀疏注意力、KV 缓存压缩、剪枝量化与幻觉诊断提供统一的结构化依据，具备直接复现与工程落地的条件。

版本：对外学术交流版 1.0

日期：2026年8月

说明：本文档为对外学术交流用途。文中全部阈值为可调工程参数，仅使用通用图论与线性代数记号。